

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5**



CONNECT TO THE INTERNET

Oleh:

Arya Arrozza Ridho Syaputra

NIM. 2410817210010

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet Sederhana ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Arya Arrozza Ridho Syaputra
NIM : 2410817210010

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	4
DAFTAR TABEL	5
SOAL	6
A. Source Code	7
B. Output Program.....	42
C. Pembahasan.....	46
Tautan GIT	58

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Homepage (en)	42
Gambar 2. Screenshot Hasil Detail Page (en)	43
Gambar 3. Screenshot Hasil Homepage (id)	43
Gambar 4. Screenshot Hasil Movie List Now Playing (id)	44
Gambar 5. Screenshot Hasil Detail Page (id)	44
Gambar 6. Screenshot Hasil Movie List No Internet (en)	45
Gambar 7. Screenshot Hasil Movie List No Internet (id)	45

DAFTAR TABEL

Tabel 1. Source Code ScrollableApplication.kt.....	7
Tabel 2. Source Code MainActivity.kt	7
Tabel 3. Source Code ScrollableApp.kt.....	8
Tabel 4. Source Code ScrollableViewModel.Kt Compose.....	10
Tabel 5. Source Code ScrollableUiState.kt.....	14
Tabel 6. Source Code ScrollableUiState.kt.....	14
Tabel 7. Source Code DetailScreen.kt	15
Tabel 8. Source Code SettingScreen.kt.....	18
Tabel 9. Source Code HomeScreen.kt	21
Tabel 10. Source Code AppPreferencesRepository.kt.....	30
Tabel 11. Source Code MovieDao.kt.....	31
Tabel 12. Source Code MovieEntity.kt.....	32
Tabel 13. Source Code MovieDatabase.kt.....	33
Tabel 14. Source Code TmdbApiService.kt	34
Tabel 15. Source Code MovieDto.kt.....	34
Tabel 16. Source Code ApiResponse.kt.....	35
Tabel 17. Source Code ErrorType.kt	35
Tabel 18. Source Code MovieResponse.kt	35
Tabel 19. Source Code NetworkModule.kt.....	36
Tabel 20. Source Code IMovieRepository.kt.....	37
Tabel 21. Source Code MovieRepository.kt.....	38

SOAL

Soal Praktikum:

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- b. Gunakan KotlinX Serialization sebagai library JSON.
- c. Gunakan library seperti Coil atau Glide untuk image loading.
- d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API:
<https://developer.themoviedb.org/docs/getting-started>
- e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
- f. Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
- g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

Tabel 1. Source Code ScrollableApplication.kt

1	package com.example.scrollablemodul3
2	
3	import android.app.Application
4	import
	com.example.scrollablemodul3.model.data.datastore.AppPre
	ferencesRepository
5	import
	com.example.scrollablemodul3.model.data.local.MovieDatab
	ase
6	import
	com.example.scrollablemodul3.model.data.repository.IMovi
	eRepository
7	import
	com.example.scrollablemodul3.model.data.repository.Movie
	Repository
8	import timber.log.Timber
9	
10	class ScrollableApplication : Application() {
11	val database by lazy {
	MovieDatabase.getDatabase(this) }
12	val preferencesRepository by lazy {
	AppPreferencesRepository(this) }
13	
14	val movieRepository: IMovieRepository by lazy {
15	MovieRepository(movieDao = database.movieDao())
16	}
17	
18	override fun onCreate() {
19	super.onCreate()
20	Timber.plant(Timber.DebugTree())
21	}
22	}

Tabel 2. Source Code MainActivity.kt

1	package com.example.scrollablemodul3
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import
	com.example.scrollablemodul3.ui.theme.ScrollableModul3Th
	eme
8	

9	class MainActivity : AppCompatActivity() {
10	override fun onCreate(savedInstanceState: Bundle?) {
11	super.onCreate(savedInstanceState)
12	enableEdgeToEdge()
13	setContent {
14	ScrollableModul3Theme {
15	ScrollableApp()
16	}
17	}
18	}
19	}

Tabel 3. Source Code ScrollableApp.kt

1	package com.example.scrollablemodul3
2	
3	import android.app.Activity
4	import android.content.Intent
5	import androidx.appcompat.app.AppCompatActivity
6	import androidx.compose.runtime.Composable
7	import androidx.compose.runtime.LaunchedEffect
8	import androidx.compose.runtime.getValue
9	import androidx.compose.ui.Modifier
10	import androidx.compose.ui.platform.LocalContext
11	import androidx.core.net.toUri
12	import
	androidx.lifecycle.compose.collectAsStateWithLifecycle
13	import androidx.lifecycle.viewmodel.compose.viewModel
14	import androidx.navigation.NavHostController
15	import androidx.navigation.compose.NavHost
16	import androidx.navigation.compose.composable
17	import androidx.navigation.compose.rememberNavController
18	import com.example.scrollablemodul3.ui.scrollable.screen
	.DetailScreen
19	import com.example.scrollablemodul3.ui.scrollable.screen
	.HomeScreen
20	import com.example.scrollablemodul3.ui.scrollable
	.ScrollableViewModel
21	import com.example.scrollablemodul3.ui.scrollable.screen
	.SettingScreen
22	
23	enum class ScrollableScreen { Home, Details, Settings }
24	
25	@Composable
26	fun ScrollableApp(
27	navController: NavHostController =
	rememberNavController())


```

28 ) {
29     val defaultLocale =
AppCompatDelegate.getApplicationLocales().get(0)?.language
?: "en"
30     val app = LocalContext.current.applicationContext as
ScrollableApplication
31     val viewModel: ScrollableViewModel = viewModel(
32         factory = ScrollableViewModel.Factory(
33             initialLocale = defaultLocale,
34             repository = app.movieRepository,
35             preferencesRepository =
app.preferencesRepository
36         )
37     )
38     val uiState by
viewModel.uiState.collectAsStateWithLifecycle()
39     val context = LocalContext.current
40
41     NavHost(
42         navController = navController,
43         startDestination = ScrollableScreen.Home.name,
44     ) {
45         composable(route = ScrollableScreen.Home.name) {
46             val activity = context as Activity
47             HomeScreen(
48                 uiState = uiState,
49                 onDetailClick = { index ->
50
navController.navigate("${ScrollableScreen.Details.name}
/$index")
51                 },
52                 onIntentClick = { url ->
53
context.startActivity(Intent(Intent.ACTION_VIEW,
url.toUri()))
54                 viewModel.onIntentButtonClicked()
55                 },
56                 onSettingsClick = {
navController.navigate(ScrollableScreen.Settings.name)
},
57                 onRefresh = { viewModel.refresh() },
58                 onExit = { activity.finish() }
59             )
60         }

```

61	composable(route =
	"\${ScrollableScreen.Details.name}/{itemId}") {
	backStackEntry ->
62	val index =
	backStackEntry.arguments?.getString("itemId")?.toIntOrNull()
	?: 0
63	val item = uiState.movies.getOrNull(index)
64	LaunchedEffect(index) {
65	viewModel.updateCurrentItem(index)
66	}
67	if (item != null) {
68	DetailScreen(
69	item = item,
70	modifier = Modifier,
71	onBackClick = {
	navController.navigateUp() }
72	)
73	}
74	}
75	composable(route =
	ScrollableScreen.Settings.name) {
76	SettingScreen(
77	modifier = Modifier,
78	onBackClick = {
	navController.navigateUp() },
79	onLocaleChange = { locale ->
	viewModel.updateLocale(locale) },
80	selectedLocale = uiState.selectedLocale
81	)
82	}
83	}
84	}

Tabel 4. Source Code ScrollableViewModel.Kt Compose

1	package com.example.scrollablemodul3.ui.scrollable
2	
3	import androidx.appcompat.app.AppCompatActivityDelegate
4	import androidx.core.os.LocaleListCompat
5	import androidx.lifecycle.ViewModel
6	import androidx.lifecycle.ViewModelProvider
7	import androidx.lifecycle.viewModelScope
8	import androidx.lifecycle.viewmodel.initializer
9	import androidx.lifecycle.viewmodel.viewModelFactory
10	import com.example.scrollablemodul3.R

```

11 import com.example.scrollablemodul3.model.data.datastore
12     .AppPreferencesRepository
13 import com.example.scrollablemodul3.model.data.remote
14     .ApiResponse
15 import
16     com.example.scrollablemodul3.model.data.remote.ErrorType
17 import com.example.scrollablemodul3.model.data
18     .repository.IMovieRepository
19 import kotlinx.coroutines.Job
20 import kotlinx.coroutines.flow.MutableStateFlow
21 import kotlinx.coroutines.flow.StateFlow
22 import kotlinx.coroutines.flow.asStateFlow
23 import kotlinx.coroutines.flow.update
24 import kotlinx.coroutines.launch
25 import timber.log.Timber
26
27 class ScrollableViewModel(
28     initialLocale: String,
29     private val repository: IMovieRepository,
30     private val preferencesRepository:
31     AppPreferencesRepository
32 ) : ViewModel() {
33     private val _uiState =
34     MutableStateFlow(ScrollableUiState(selectedLocale =
35     initialLocale))
36     val uiState: StateFlow<ScrollableUiState> =
37     _uiState.asStateFlow()
38     private var fetchJob: Job? = null
39
40     init {
41         viewModelScope.launch {
42             preferencesRepository.selectedLocale.collect
43             { locale ->
44                 val appLocale = mapLocale(locale)
45                 val isNewLanguage = appLocale !=
46                 _uiState.value.selectedLanguage
47
48                 _uiState.update { it.copy(
49                     selectedLocale = locale,
50                     selectedLanguage = appLocale,
51                     movies = emptyList(),
52                     isLoading = true
53                 ) }
54
55                 loadMovies(language = appLocale,
56                     forceRefresh = isNewLanguage)

```

```

46         }
47     }
48 }
49
50 fun mapLocale(locale: String) = when (locale) {
51     "in" -> "id"
52     else -> "en-US"
53 }
54
55 private fun loadMovies(
56     language: String =
57     _uiState.value.selectedLanguage,
58     forceRefresh: Boolean = false
59 ) {
60     fetchJob?.cancel()
61
62     fetchJob = viewModelScope.launch {
63         repository.getPopularMovies(language,
64         forceRefresh).collect { response ->
65             when (response) {
66                 is ApiResponse.Loading -> {
67                     _uiState.update {
68                         it.copy(isLoading = true, errorMessage =
69                         ErrorMessage.None) }
70                     }
71                     is ApiResponse.Success -> {
72                         _uiState.update { it.copy(
73                             movies = response.data,
74                             isLoading = false,
75                             errorMessage =
76                             ErrorMessage.None
77                             ) }
78                     }
79                     is ApiResponse.Error -> {
80                         val message = when
81                         (response.errorType) {
82                             ErrorType.NO_INTERNET ->
83                             R.string.error_no_internet
84                             ErrorType.NO_RESULTS ->
85                             R.string.error_no_movies
86                             ErrorType.SERVER_OR_CLIENT_ERROR ->
87                             R.string.error_server_client
88                             ErrorType.TIMEOUT ->
89                             R.string.error_timeout

```

```

80         else ->
81         R.string.error_unexpected
82         }
83         _uiState.update { it.copy(
84             errorMessage =
85             ErrorMessage.ErrorMessageRes(message),
86             isLoading = false
87         ) }
88     }
89 }
90 }
91
92 fun refresh() {
93     loadMovies(forceRefresh = true)
94 }
95
96 fun updateCurrentItem(index: Int) {
97     val selectedItem =
98     _uiState.value.movies.getOrNull(index)
99     Timber.d("Selected at index $index of
100     movieTitle: ${selectedItem?.title}")
101
102     _uiState.value = _uiState.value.copy(
103         currentItemIndex = index
104     )
105 }
106
107 fun updateLocale(locale: String) {
108     _uiState.value = _uiState.value.copy(
109         selectedLocale = locale
110     )
111     val appLocale =
112     LocaleListCompat.forLanguageTags(locale)
113     AppCompatActivity.setApplicationLocales(appLocale)
114
115     viewModelScope.launch {
116         preferencesRepository.saveLocale(locale)
117     }
118 }
119
120 fun onIntentButtonClicked() {
121     Timber.d("Intent button clicked")
122 }

```

120	
121	companion object {
122	fun Factory(
123	initialLocale: String,
124	repository: IMovieRepository,
125	preferencesRepository:
	AppPreferencesRepository
126	): ViewModelProvider.Factory = viewModelFactory
	{
127	initializer {
128	ScrollableViewModel(initialLocale,
	repository, preferencesRepository)
129	}
130	}
131	}
132	}

Tabel 5. Source Code ScrollableUiState.kt

1	package com.example.scrollablemodul3.ui.scrollable
2	
3	import com.example.scrollablemodul3.model.data.local
	.entity.MovieEntity
4	
5	data class ScrollableUiState(
6	val movies: List<MovieEntity> = emptyList(),
7	val isLoading: Boolean = false,
8	val errorMessage: ErrorMessage = ErrorMessage.None,
9	val currentItemIndex: Int = 0,
10	val selectedLocale: String = "en",
11	val selectedLanguage: String = "en-US"
12	)

Tabel 6. Source Code ScrollableUiState.kt

1	package com.example.scrollablemodul3.ui.scrollable
2	
3	import androidx.annotation.StringRes
4	
5	sealed interface ErrorMessage {
6	data class ErrorMessageRes(@StringRes val
	messageRes: Int) : ErrorMessage
7	data object None : ErrorMessage
8	}

Tabel 7. Source Code DetailScreen.kt

1	package
	com.example.scrollablemodul3.ui.scrollable.screen
2	
3	import androidx.compose.foundation.BorderStroke
4	import androidx.compose.foundation.layout.Column
5	import androidx.compose.foundation.layout.Row
6	import androidx.compose.foundation.layout.Spacer
7	import androidx.compose.foundation.layout.fillMaxWidth
8	import androidx.compose.foundation.layout.height
9	import androidx.compose.foundation.layout.padding
10	import androidx.compose.foundation.layout.size
11	import androidx.compose.foundation.layout.width
12	import androidx.compose.foundation.rememberScrollState
13	import
	androidx.compose.foundation.shape.RoundedCornerShape
14	import androidx.compose.foundation.verticalScroll
15	import androidx.compose.material.icons.Icons
16	import
	androidx.compose.material.icons.automirrored.filled.Arro
	wBack
17	import androidx.compose.material3.Card
18	import androidx.compose.material3.CardDefaults
19	import androidx.compose.material3.CenterAlignedTopAppBar
20	import
	androidx.compose.material3.ExperimentalMaterial3Api
21	import androidx.compose.material3.Icon
22	import androidx.compose.material3.IconButton
23	import androidx.compose.material3.MaterialTheme
24	import androidx.compose.material3.Scaffold
25	import androidx.compose.material3.Text
26	import androidx.compose.runtime.Composable
27	import androidx.compose.ui.Alignment
28	import androidx.compose.ui.Modifier
29	import androidx.compose.ui.layout.ContentScale
30	import androidx.compose.ui.platform.LocalContext
31	import androidx.compose.ui.res.painterResource
32	import androidx.compose.ui.res.stringResource
33	import androidx.compose.ui.text.style.TextAlign
34	import androidx.compose.ui.unit.dp
35	import coil3.compose.AsyncImage
36	import coil3.request.ImageRequest
37	import coil3.request.crossfade
38	import coil3.request.error
39	import coil3.request.fallback
40	import coil3.request.placeholder

```

41 import com.example.scrollablemodul3.R
42 import
   com.example.scrollablemodul3.model.data.local.entity.MovieEntity
43
44 @OptIn(ExperimentalMaterial3Api::class)
45 @Composable
46 fun DetailScreen(
47     item: MovieEntity,
48     onBackPressed: () -> Unit,
49     modifier: Modifier = Modifier
50 ){
51     val context = LocalContext.current
52     Scaffold(
53         topBar = {
54             CenterAlignedTopAppBar(
55                 title = {
56                     Text(text = item.title)
57                 },
58                 navigationIcon = {
59                     IconButton(onClick = onBackPressed) {
60                         Icon(
61                             imageVector =
62                             Icons.AutoMirrored.Filled.ArrowBack,
63                             contentDescription =
64                             stringResource(R.string.back_button)
65                         )
66                     }
67                 }
68             ) { innerPadding ->
69                 Column(
70                     modifier = modifier
71                     .padding(innerPadding)
72                     .verticalScroll(rememberScrollState())
73                 ) {
74                     AsyncImage(
75                         model = ImageRequest.Builder(context)
76                             .data(item.fullPosterUrl)
77                             .crossfade(true)
78
79                         .placeholder(R.drawable.image_loading)
80                         .error(R.drawable.image_error)
81                         .fallback(R.drawable.image_error)
82                         .build(),

```



```

82         contentDescription = item.title,
83         contentScale = ContentScale.Crop,
84         modifier = Modifier
85             .fillMaxWidth()
86             .height(450.dp)
87     )
88
89     Column(modifier = Modifier.padding(16.dp)) {
90         Text(
91             text = item.title,
92             style =
93                 MaterialTheme.typography.headlineMedium
94         )
95         Spacer(modifier = Modifier.height(4.dp))
96         Row(verticalAlignment =
97             Alignment.CenterVertically) {
98             Icon(
99                 painter = painterResource(id =
100                     R.drawable.star),
101                 contentDescription =
102                     stringResource(R.string.rating_format),
103                 modifier = Modifier.size(20.dp)
104             )
105             Spacer(modifier =
106                 Modifier.width(4.dp))
107             Text(
108                 text =
109                     stringResource(R.string.rating_format,
110                         item.voteAverage),
111                 style =
112                     MaterialTheme.typography.titleMedium
113             )
114             Spacer(modifier =
115                 Modifier.width(16.dp))
116             Text(
117                 text = item.releaseDate.ifEmpty
118                 { stringResource(R.string.unknown) },
119                 style =
120                     MaterialTheme.typography.bodyLarge
121             )
122         }
123     }
124
125     Card(
126         modifier = Modifier
127             .fillMaxWidth()
128             .padding(top = 16.dp),

```

117	elevation =
118	CardDefaults.cardElevation(defaultElevation = 4.dp),
119	shape = RoundedCornerShape(8.dp),
	border = BorderStroke(1.dp,
	MaterialTheme.colorScheme.outline)
120	) {
121	Column(modifier =
	Modifier.padding(16.dp)) {
122	Text(
123	text =
	stringResource(R.string.overview_title),
124	style =
	MaterialTheme.typography.titleLarge
125	)
126	Spacer(modifier =
	Modifier.height(8.dp))
127	Text(
128	text = item.overview.ifEmpty
	{ stringResource(R.string.no_overview) },
129	style =
	MaterialTheme.typography.bodyMedium,
130	textAlign =
	TextAlign.Justify
131	)
132	}
133	}
134	}
135	}
136	}
137	}

Tabel 8. Source Code SettingScreen.kt

1	package com.example.scrollablemodul3.ui.screen
2	
3	import androidx.compose.foundation.layout.Column
4	import androidx.compose.foundation.layout.Row
5	import androidx.compose.foundation.layout.fillMaxWidth
6	import androidx.compose.foundation.layout.padding
7	import androidx.compose.foundation.rememberScrollState
8	import androidx.compose.foundation.verticalScroll
9	import androidx.compose.material.icons.Icons
10	import androidx.compose.material.icons.automirrored.filled.ArrowBack
11	import androidx.compose.material3.CenterAlignedTopAppBar
12	import
	androidx.compose.material3.ExperimentalMaterial3Api

```

13 import androidx.compose.material3.Icon
14 import androidx.compose.material3.IconButton
15 import androidx.compose.material3.MaterialTheme
16 import androidx.compose.material3.RadioButton
17 import androidx.compose.material3.Scaffold
18 import androidx.compose.material3.Text
19 import androidx.compose.runtime.Composable
20 import androidx.compose.ui.Alignment
21 import androidx.compose.ui.Modifier
22 import androidx.compose.ui.res.stringResource
23 import androidx.compose.ui.tooling.preview.Preview
24 import androidx.compose.ui.unit.dp
25 import com.example.scrollablemodul3.R
26
27 @OptIn(ExperimentalMaterial3Api::class)
28 @Composable
29 fun SettingScreen(
30     modifier: Modifier = Modifier,
31     onBackClick: () -> Unit,
32     onLocaleChange: (String) -> Unit,
33     selectedLocale: String
34 ){
35     Scaffold(
36         topBar = {
37             CenterAlignedTopAppBar(
38                 title = {
39                     Text(text =
stringResource(R.string.setting_button))
40                 },
41                 navigationIcon = {
42                     IconButton(onClick = onBackClick) {
43                         Icon(
44                             imageVector =
Icons.AutoMirrored.Filled.ArrowBack,
45                             contentDescription =
stringResource(R.string.back_button)
46                         )
47                     }
48                 }
49             )
50         }
51     ) { innerPadding ->
52         Column(
53             modifier = modifier
54                 .padding(innerPadding)
55                 .padding(16.dp)

```

```

56         .fillMaxWidth()
57         .verticalScroll(rememberScrollState())
58     ) {
59         Text(
60             text =
stringResource(R.string.setting_header),
61             style =
MaterialTheme.typography.headlineSmall,
62             modifier = Modifier.padding(vertical =
8.dp)
63         )
64
65         Row(
66             verticalAlignment =
Alignment.CenterVertically,
67             modifier = Modifier.fillMaxWidth()
68         ) {
69             RadioButton(
70                 selected = selectedLocale == "en",
71                 onClick = { onLocaleChange("en") }
72             )
73             Text(text =
stringResource(R.string.english_button))
74         }
75         Row(
76             verticalAlignment =
Alignment.CenterVertically,
77             modifier = Modifier.fillMaxWidth()
78         ) {
79             RadioButton(
80                 selected = selectedLocale == "in",
81                 onClick = { onLocaleChange("in") }
82             )
83             Text(text =
stringResource(R.string.indonesian_button))
84         }
85     }
86 }
87
88
89 @Preview
90 @Composable
91 fun SettingScreenLayout() {
92     SettingScreen(
93         modifier = Modifier,
94         onBackClick = {},

```

95	onLocaleChange = {},
96	selectedLocale = "en"
97	)
98	}

Tabel 9. Source Code HomeScreen.kt

1	package
	com.example.scrollablemodul3.ui.scrollable.screen
2	
3	import androidx.compose.foundation.BorderStroke
4	import
	androidx.compose.foundation.gestures.snapping.rememberSnapFlingBehavior
5	import androidx.compose.foundation.layout.Arrangement
6	import androidx.compose.foundation.layout.Box
7	import androidx.compose.foundation.layout.Column
8	import androidx.compose.foundation.layout.PaddingValues
9	import androidx.compose.foundation.layout.Row
10	import androidx.compose.foundation.layout.Spacer
11	import androidx.compose.foundation.layout.fillMaxSize
12	import androidx.compose.foundation.layout.fillMaxWidth
13	import androidx.compose.foundation.layout.height
14	import androidx.compose.foundation.layout.padding
15	import androidx.compose.foundation.layout.size
16	import androidx.compose.foundation.layout.width
17	import androidx.compose.foundation.lazy.LazyColumn
18	import androidx.compose.foundation.lazy.LazyRow
19	import androidx.compose.foundation.lazy.itemsIndexed
20	import
	androidx.compose.foundation.lazy.rememberLazyListState
21	import
	androidx.compose.foundation.shape.RoundedCornerShape
22	import androidx.compose.material.icons.Icons
23	import
	androidx.compose.material.icons.automirrored.filled.ExitToApp
24	import androidx.compose.material.icons.filled.Refresh
25	import androidx.compose.material.icons.filled.Settings
26	import androidx.compose.material3.Button
27	import androidx.compose.material3.Card
28	import androidx.compose.material3.CardDefaults
29	import
	androidx.compose.material3.CircularProgressIndicator
30	import
	androidx.compose.material3.ExperimentalMaterial3Api
31	import androidx.compose.material3.Icon

```

32 import androidx.compose.material3.IconButton
33 import
   androidx.compose.material3.LinearProgressIndicator
34 import androidx.compose.material3.MaterialTheme
35 import androidx.compose.material3.Scaffold
36 import androidx.compose.material3.Text
37 import androidx.compose.material3.TopAppBar
38 import androidx.compose.runtime.Composable
39 import androidx.compose.ui.Alignment
40 import androidx.compose.ui.Modifier
41 import androidx.compose.ui.draw.clip
42 import androidx.compose.ui.layout.ContentScale
43 import androidx.compose.ui.platform.LocalContext
44 import androidx.compose.ui.res.painterResource
45 import androidx.compose.ui.res.stringResource
46 import androidx.compose.ui.text.style.TextOverflow
47 import androidx.compose.ui.unit.dp
48 import coil3.compose.AsyncImage
49 import coil3.request.ImageRequest
50 import coil3.request.crossfade
51 import coil3.request.error
52 import coil3.request.fallback
53 import coil3.request.placeholder
54 import com.example.scrollablemodul3.R
55 import
   com.example.scrollablemodul3.model.data.local.entity.MovieEntity
56 import
   com.example.scrollablemodul3.ui.scrollable.ErrorMessage
57 import
   com.example.scrollablemodul3.ui.scrollable.ScrollableUiState
58
59 @OptIn(ExperimentalMaterial3Api::class)
60 @Composable
61 fun HomeAppBar(
62     onSettingClick: () -> Unit,
63     onRefresh: () -> Unit,
64     onExit: () -> Unit
65 ) {
66     TopAppBar(
67         title = {
68             Text(stringResource(R.string.head_title))
69         },
70         navigationIcon = {
71             IconButton(onClick = onExit) {

```

```

72             Icon(
73                 imageVector =
Icons.AutoMirrored.Filled.ExitToApp,
74                 contentDescription =
stringResource(R.string.back_button)
75             )
76         },
77     },
78     actions = {
79         IconButton(onClick = onRefresh) {
80             Icon(
81                 imageVector = Icons.Default.Refresh,
82                 contentDescription =
stringResource(R.string.refresh_button)
83             )
84         }
85         IconButton(onClick = onSettingClick) {
86             Icon(
87                 imageVector =
Icons.Default.Settings,
88                 contentDescription =
stringResource(R.string.setting_button)
89             )
90         }
91     }
92 )
93 }
94
95 @Composable
96 fun HomeScreen(
97     uiState: ScrollableUiState,
98     onDetailClick: (Int) -> Unit,
99     onIntentClick: (String) -> Unit,
100    onSettingsClick: () -> Unit,
101    onRefresh: () -> Unit,
102    onExit: () -> Unit,
103    modifier: Modifier = Modifier
104 ){
105     Scaffold(
106         topBar = {
107             HomeAppBar(
108                 onSettingClick = onSettingsClick,
109                 onRefresh = onRefresh,
110                 onExit = onExit
111             )
112         }

```

```

113     ) { innerPadding ->
114         Column(modifier =
115             modifier.padding(innerPadding)) {
116                 if (uiState.movies.isEmpty() &&
117                     uiState.isLoading) {
118                     LinearProgressIndicator(modifier =
119                         Modifier.fillMaxWidth())
120                 }
121                 Box(modifier = Modifier.fillMaxSize()) {
122                     when {
123                         uiState.movies.isEmpty() &&
124                         uiState.isLoading -> {
125                             CircularProgressIndicator(modifier =
126                                 Modifier.align(Alignment.Center))
127                             }
128                             uiState.errorMessage is
129                             ErrorMessage.ErrorMessageRes && uiState.movies.isEmpty()
130                             -> {
131                                 val error = uiState.errorMessage
132                                 Column(
133                                     modifier = Modifier
134                                         .align(Alignment.Center)
135                                         .padding(24.dp),
136                                     horizontalAlignment =
137                                     Alignment.CenterHorizontally
138                                 ) {
139                                     Text(
140                                         text =
141                                         stringResource(error.messageRes),
142                                         style =
143                                         MaterialTheme.typography.bodyMedium,
144                                         color =
145                                         MaterialTheme.colorScheme.error
146                                     )
147                                     Spacer(modifier =
148                                         Modifier.padding(8.dp))
149                                     Button(onClick = onRefresh)
150                                     {
151                                         Text(text =
152                                             stringResource(R.string.refresh_button))
153                                     }
154                                 }
155                             }
156                         }
157                     }
158                 }
159             }
160         }
161     }

```



```

144         else -> {
145             LazyColumn(
146                 modifier =
Modifier.fillMaxSize(),
147                 contentPadding =
PaddingValues(start = 8.dp, bottom = 16.dp, end = 8.dp)
148             ) {
149                 item {
150                     ItemCarousel(
151                         items =
uiState.movies,
152                         onDetailClick = {
index -> onDetailClick(index) }
153                     )
154                 }
155                 itemsIndexed(uiState.movies,
key = { _, movie -> movie.id }) { index, item ->
156                     ItemCard(
157                         item = item,
158                         onDetailClick = {
onDetailClick(index) },
159                         onIntentClick = {
onIntentClick("https://www.themoviedb.org/movie/${item.i
d}") }
160                     )
161                 }
162             }
163         }
164     }
165 }
166 }
167 }
168 }
169
170 @Composable
171 fun ItemCard(
172     item: MovieEntity,
173     onDetailClick: () -> Unit,
174     onIntentClick: () -> Unit,
175     modifier: Modifier = Modifier
176 ) {
177     val context = LocalContext.current
178     Card(
179         modifier = modifier
180             .padding(8.dp)
181             .fillMaxWidth(),

```

```

182         shape = MaterialTheme.shapes.medium,
183         elevation =
CardDefaults.cardElevation(defaultElevation = 4.dp),
184         border = BorderStroke(1.dp,
MaterialTheme.colorScheme.outline)
185     ) {
186         Row(modifier = Modifier.padding(16.dp)) {
187             AsyncImage(
188                 model = ImageRequest.Builder(context)
189                     .data(item.fullPosterUrl)
190                     .crossfade(true)
191
.placeholder(R.drawable.image_loading)
192                 .error(R.drawable.image_error)
193                 .fallback(R.drawable.image_error)
194                 .build(),
195                 contentDescription = item.title,
196                 contentScale = ContentScale.Crop,
197                 modifier = Modifier
198                     .width(100.dp)
199                     .height(140.dp)
200                     .clip(RoundedCornerShape(16.dp))
201             )
202             Column(
203                 modifier = Modifier
204                     .padding(start = 16.dp)
205                     .weight(1f)
206             ) {
207                 Text(
208                     text = item.title,
209                     style =
MaterialTheme.typography.headlineSmall,
210                     maxLines = 2,
211                     overflow = TextOverflow.Ellipsis
212                 )
213                 Spacer(modifier = Modifier.height(4.dp))
214                 Row(verticalAlignment =
Alignment.CenterVertically) {
215                     Icon(
216                         painter = painterResource(id =
R.drawable.star),
217                         contentDescription =
stringResource(R.string.rating_format),
218                         modifier = Modifier.size(16.dp)
219                     )

```

```

220         Spacer(modifier =
221         Modifier.width(4.dp))
222         Text(
223             text =
224             stringResource(R.string.rating_format,
225             item.voteAverage),
226             style =
227             MaterialTheme.typography.bodyMedium
228         )
229     }
230     Text(
231         text = item.releaseDate.ifEmpty {
232         stringResource(R.string.unknown) },
233         style =
234         MaterialTheme.typography.bodyMedium
235     )
236     Row(
237         modifier = Modifier
238         .padding(top = 16.dp)
239         .fillMaxWidth(),
240         horizontalArrangement =
241         Arrangement.End,
242     ) {
243         Button(
244             onClick = onIntentClick,
245             contentPadding =
246             PaddingValues(horizontal = 12.dp)
247         ) {
248             Text(
249                 text =
250                 stringResource(R.string.tmdb_button),
251                 style =
252                 MaterialTheme.typography.bodySmall
253             )
254         }
255         Spacer(modifier =
256         Modifier.width(8.dp))
257         Button(
258             onClick = onDetailClick,
259             contentPadding =
260             PaddingValues(horizontal = 12.dp)
261         ) {
262             Text(
263                 text =
264                 stringResource(R.string.detail_button),

```

```

253         style =
MaterialTheme.typography.bodySmall
254     )
255     }
256     }
257     }
258     }
259     }
260 }
261
262 @Composable
263 fun CarouselCard(
264     item: MovieEntity,
265     onDetailClick: () -> Unit,
266     modifier: Modifier = Modifier
267 ){
268     val context = LocalContext.current
269     Card(
270         onClick = onDetailClick,
271         modifier = modifier.padding(8.dp),
272         shape = MaterialTheme.shapes.medium,
273         elevation =
CardDefaults.cardElevation(defaultElevation = 4.dp),
274         border = BorderStroke(1.dp,
MaterialTheme.colorScheme.outline),
275         colors = CardDefaults.cardColors(
276             containerColor =
MaterialTheme.colorScheme.surfaceContainer
277         )
278     ) {
279         Box(
280             modifier = Modifier
281                 .fillMaxWidth()
282                 .height(200.dp)
283         ) {
284             AsyncImage(
285                 model = ImageRequest.Builder(context)
286                     .data(item.fullBackdropUrl)
287                     .crossfade(true)
288
.placeholder(R.drawable.image_loading)
289                 .error(R.drawable.image_error)
290                 .fallback(R.drawable.image_error)
291                 .build(),
292                 contentDescription = item.title,
293                 contentScale = ContentScale.Crop,

```

```

294         modifier = Modifier
295             .fillMaxSize()
296             .clip(RoundedCornerShape(16.dp))
297     )
298
299     Box(
300         modifier = Modifier
301             .align(Alignment.BottomStart)
302             .fillMaxWidth()
303             .padding(16.dp)
304     ) {
305         Text(
306             text = item.title,
307             style =
MaterialTheme.typography.headlineSmall,
308             color =
MaterialTheme.colorScheme.onSurfaceVariant,
309             maxLines = 1,
310             overflow = TextOverflow.Ellipsis
311         )
312     }
313 }
314 }
315 }
316
317 @Composable
318 fun ItemCarousel(
319     items: List<MovieEntity>,
320     onDetailClick: (Int) -> Unit,
321 ) {
322     val listState = rememberLazyListState()
323     LazyRow(
324         state = listState,
325         flingBehavior =
rememberSnapFlingBehavior(listState)
326     ) {
327         itemsIndexed(items) { index, item ->
328             CarouselCard(
329                 item = item,
330                 onDetailClick = { onDetailClick(index)
},
331                 modifier = Modifier.fillParentMaxWidth()
332             )
333         }
334     }
335 }

```

Tabel 10. Source Code AppPreferencesRepository.kt

```

1 package
  com.example.scrollablemodul3.model.data.datastore
2
3 import android.content.Context
4 import androidx.datastore.core.DataStore
5 import androidx.datastore.preferences.core.Preferences
6 import androidx.datastore.preferences.core.edit
7 import
  androidx.datastore.preferences.core.emptyPreferences
8 import
  androidx.datastore.preferences.core.stringPreferencesKey
9 import
  androidx.datastore.preferences.preferencesDataStore
10 import kotlinx.coroutines.flow.Flow
11 import kotlinx.coroutines.flow.catch
12 import kotlinx.coroutines.flow.map
13 import timber.log.Timber
14 import java.io.IOException
15
16 private val Context.dataStore: DataStore<Preferences> by
  preferencesDataStore(name = "app_preferences")
17
18 class AppPreferencesRepository(private val context:
  Context) {
19     companion object {
20         private val KEY_LOCALE =
  stringPreferencesKey("selected_locale")
21     }
22
23     val selectedLocale: Flow<String> =
  context.dataStore.data
24         .catch { exception ->
25             if (exception is IOException) {
26                 Timber.e(exception, "Error reading
  locale preference")
27                 emit(emptyPreferences())
28             } else {
29                 throw exception
30             }
31         }
32         .map { preferences -> preferences[KEY_LOCALE] ?:
  "en" }
33
34     suspend fun saveLocale(locale: String) {
35         context.dataStore.edit { preferences ->

```

36	preferences[KEY_LOCALE] = locale
37	}
38	}
39	}

Tabel 11. Source Code MovieDao.kt

1	package
	com.example.scrollablemodul3.model.data.local.dao
2	
3	import androidx.room.Dao
4	import androidx.room.Insert
5	import androidx.room.OnConflictStrategy
6	import androidx.room.Query
7	import com.example.scrollablemodul3.model.data.local.
	entity.MovieEntity
8	import kotlinx.coroutines.flow.Flow
9	
10	@Dao
11	interface MovieDao {
12	@Query("SELECT * FROM movies WHERE category =
	:category AND language = :language ORDER BY voteAverage
	DESC")
13	fun getMoviesByCategoryAndLanguage(category: String,
	language: String): Flow<List<MovieEntity>>
14	
15	@Insert(onConflict = OnConflictStrategy.REPLACE)
16	suspend fun insertAll(movies: List<MovieEntity>)
17	
18	@Query("DELETE FROM movies WHERE category =
	:category AND language = :language")
19	suspend fun deleteByCategoryAndLanguage(category:
	String, language: String)
20	
21	@Query("SELECT cachedAt FROM movies WHERE category =
	:category AND language = :language LIMIT 1")
22	suspend fun getLastCachedTime(category: String,
	language: String): Long?
23	
24	@Query("SELECT * FROM movies WHERE category =
	:category ORDER BY voteAverage DESC")
25	fun getAnyMoviesByCategory(category: String):
	Flow<List<MovieEntity>>
26	}

Tabel 12. Source Code MovieEntity.kt

```
1 package
  com.example.scrollablemodul3.model.data.local.entity
2
3 import androidx.room.Entity
4 import androidx.room.Index
5
6 @Entity(
7     tableName = "movies",
8     primaryKeys = ["id", "language"],
9     indices = [Index(value = ["category", "language"])]
10 )
11 data class MovieEntity(
12     val id: Int,
13     val title: String,
14     val overview: String,
15     val posterPath: String?,
16     val backdropPath: String?,
17     val voteAverage: Double,
18     val voteCount: Int,
19     val releaseDate: String,
20     val originalLanguage: String,
21     val popularity: Double,
22     val category: String,
23     val language: String,
24     val cachedAt: Long = System.currentTimeMillis()
25 ) {
26     val fullPosterUrl: String?
27     get() =
28         if (posterPath.isNullOrEmpty()) null
29         else
30             "https://image.tmdb.org/t/p/w500$posterPath"
31     val fullBackdropUrl: String?
32     get() =
33         if (backdropPath.isNullOrEmpty()) null
34         else
35             "https://image.tmdb.org/t/p/w780$backdropPath"
36 }
```


Tabel 13. Source Code MovieDatabase.kt

```

1 package com.example.scrollablemodul3.model.data.local
2
3 import android.content.Context
4 import androidx.room.Database
5 import androidx.room.Room
6 import androidx.room.RoomDatabase
7 import
8 com.example.scrollablemodul3.model.data.local.dao.MovieD
9 ao
10 import
11 com.example.scrollablemodul3.model.data.local.entity.Mov
12 ieEntity
13
14 @Database(
15     entities = [MovieEntity::class],
16     version = 1,
17     exportSchema = false
18 )
19 abstract class MovieDatabase : RoomDatabase() {
20     abstract fun movieDao(): MovieDao
21
22     companion object {
23         @Volatile
24         private var INSTANCE: MovieDatabase? = null
25
26         fun getDatabase(context: Context): MovieDatabase
27         =
28             INSTANCE ?: synchronized(this) {
29                 Room.databaseBuilder(
30                     context.applicationContext,
31                     MovieDatabase::class.java,
32                     "movie_scrollable_db"
33                 )
34                 .build().also {
35                     INSTANCE = it
36                 }
37             }
38     }
39 }

```

Tabel 14. Source Code TmdbApiService.kt

1	package
2	com.example.scrollablemodul3.model.data.remote.api
3	import com.example.scrollablemodul3.model.data.remote
4	.MovieResponse
5	import retrofit2.Response
6	import retrofit2.http.GET
7	import retrofit2.http.Query
8	interface TmdbApiService {
9	@GET("movie/popular")
10	suspend fun getPopularMovies(
11	@Query("page") page: Int = 1,
12	@Query("language") language: String
13	): Response<MovieResponse>
14	
15	@GET("movie/now_playing")
16	suspend fun getNowPlayingMovies(
17	@Query("page") page: Int = 1,
18	@Query("language") language: String
19	): Response<MovieResponse>
20	
21	@GET("movie/top_rated")
22	suspend fun getTopRatedMovies(
23	@Query("page") page: Int = 1,
24	@Query("language") language: String
25	): Response<MovieResponse>
26	}

Tabel 15. Source Code MovieDto.kt

1	package
2	com.example.scrollablemodul3.model.data.remote.dto
3	import kotlinx.serialization.SerialName
4	import kotlinx.serialization.Serializable
5	
6	
7	@Serializable
8	data class MovieDto(
9	val id: Int,
10	val title: String,
11	val overview: String,
12	@SerialName("poster_path") val posterPath: String? =
	null,

13	@SerializedName("backdrop_path") val backdropPath:
	String? = null,
14	@SerializedName("vote_average") val voteAverage: Double
	= 0.0,
15	@SerializedName("vote_count") val voteCount: Int = 0,
16	@SerializedName("release_date") val releaseDate: String
	= "",
17	@SerializedName("genre_ids") val genreIds: List<Int> =
	emptyList(),
18	@SerializedName("original_language") val
	originalLanguage: String = "",
19	val popularity: Double = 0.0
20	)

Tabel 16. Source Code ApiResponse.kt

1	package com.example.scrollablemodul3.model.data.remote
2	
3	sealed interface ApiResponse<out T> {
4	data class Success<out T>(val data: T) :
	ApiResponse<T>
5	data class Error(val errorType: ErrorType =
	ErrorType.UNKNOWN) : ApiResponse<Nothing>
6	data object Loading : ApiResponse<Nothing>
7	}

Tabel 17. Source Code ErrorType.kt

1	package com.example.scrollablemodul3.model.data.remote
2	
3	enum class ErrorType {
4	NO_INTERNET,
5	NO_RESULTS,
6	SERVER_OR_CLIENT_ERROR,
7	TIMEOUT,
8	UNKNOWN
9	}

Tabel 18. Source Code MovieResponse.kt

1	package com.example.scrollablemodul3.model.data.remote
2	
3	import
	com.example.scrollablemodul3.model.data.remote.dto.Movie
	Dto
4	import kotlinx.serialization.SerialName
5	import kotlinx.serialization.Serializable
6	
7	@Serializable

8	data class MovieResponse(
9	val page: Int,
10	val results: List<MovieDto>,
11	@SerializedName("total_pages") val totalPages: Int,
12	@SerializedName("total_results") val totalResults: Int
13	)

Tabel 19. Source Code NetworkModule.kt

1	package com.example.scrollablemodul3.model.data.remote
2	
3	import com.example.scrollablemodul3.BuildConfig
4	import
	com.example.scrollablemodul3.model.data.remote.api.TmdbA
	piService
5	import
	com.jakewharton.retrofit2.converter.kotlinx.serialization.asConverterFactory
6	import kotlinx.serialization.json.Json
7	import okhttp3.Interceptor
8	import okhttp3.MediaType.Companion.toMediaType
9	import okhttp3.OkHttpClient
10	import okhttp3.logging.HttpLoggingInterceptor
11	import retrofit2.Retrofit
12	import java.util.concurrent.TimeUnit
13	
14	object NetworkModule {
15	private const val BASE_URL =
	"https://api.themoviedb.org/3/"
16	
17	private val json = Json {
18	ignoreUnknownKeys = true
19	coerceInputValues = true
20	isLenient = true
21	}
22	
23	private val loggingInterceptor =
	HttpLoggingInterceptor().apply {
24	level = if (BuildConfig.DEBUG) {
25	HttpLoggingInterceptor.Level.BODY
26	} else {
27	HttpLoggingInterceptor.Level.NONE
28	}
29	}
30	
31	private val authInterceptor = Interceptor { chain ->
32	val originalRequest = chain.request()

33	val authorizedUrl =
	originalRequest.url.newBuilder()
34	.addQueryParameter("api_key",
	BuildConfig.TMDB_API_KEY)
35	.build()
36	val newRequest = originalRequest.newBuilder()
37	.url(authorizedUrl)
38	.build()
39	chain.proceed(newRequest)
40	}
41	
42	private val okHttpClient = OkHttpClient.Builder()
43	.addInterceptor(authInterceptor)
44	.addInterceptor(loggingInterceptor)
45	.connectTimeout(30, TimeUnit.SECONDS)
46	.readTimeout(30, TimeUnit.SECONDS)
47	.writeTimeout(30, TimeUnit.SECONDS)
48	.build()
49	
50	private val retrofit = Retrofit.Builder()
51	.baseUrl(BASE_URL)
52	.client(okHttpClient)
53	
	.addConverterFactory(json.asConverterFactory("applicatio
	n/json".toMediaType()))
54	.build()
55	
56	val tmdbApiService: TmdbApiService =
	retrofit.create(TmdbApiService::class.java)
57	}

Tabel 20. Source Code IMovieRepository.kt

1	package
	com.example.scrollablemodul3.model.data.repository
2	
3	import
	com.example.scrollablemodul3.model.data.local.entity.Mov
	ieEntity
4	import
	com.example.scrollablemodul3.model.data.remote.ApiRespon
	se
5	import kotlinx.coroutines.flow.Flow
6	
7	interface IMovieRepository {
8	

9	fun getPopularMovies(language: String, forceRefresh: Boolean = false):
10	Flow<ApiResponse<List<MovieEntity>>>
11	fun getNowPlayingMovies(language: String, forceRefresh: Boolean = false):
12	Flow<ApiResponse<List<MovieEntity>>>
13	fun getTopRatedMovies(language: String, forceRefresh: Boolean = false):
14	Flow<ApiResponse<List<MovieEntity>>>
15	}

Tabel 21. Source Code MovieRepository.kt

1	package
2	com.example.scrollablemodul3.model.data.repository
3	import com.example.scrollablemodul3.model.data.local.dao
4	.MovieDao
5	import com.example.scrollablemodul3.model.data.local
6	.entity.MovieEntity
7	import com.example.scrollablemodul3.model.data.remote
8	.ApiResponse
9	import
10	com.example.scrollablemodul3.model.data.remote.ErrorType
11	import com.example.scrollablemodul3.model.data
12	.remote.NetworkModule
13	import com.example.scrollablemodul3.model.data.remote
14	.api.TmdbApiService
15	import com.example.scrollablemodul3.model.data.remote
16	.dto.MovieDto
17	import com.example.scrollablemodul3.model.data
18	.remote.MovieResponse
19	import kotlinx.coroutines.flow.Flow
20	import kotlinx.coroutines.flow.FlowCollector
21	import kotlinx.coroutines.flow.first
22	import kotlinx.coroutines.flow.flow
23	import retrofit2.Response
24	import timber.log.Timber
25	import java.net.SocketTimeoutException
26	import java.net.UnknownHostException
27	
28	class MovieRepository(
29	private val apiService: TmdbApiService =
30	NetworkModule.tmdbApiService,
31	private val movieDao: MovieDao
32	): IMovieRepository {
33	companion object {

```

25         private const val CACHE_DURATION_MS = 30 * 60 *
26             1000
27         }
28         private suspend fun isCacheValid(category: String,
29             language: String): Boolean {
30             val lastCache =
31             movieDao.getLastCachedTime(category, language) ?: return
32             false
33             val age = System.currentTimeMillis() - lastCache
34             return age < CACHE_DURATION_MS
35         }
36
37         private fun MovieDto.toEntity(category: String,
38             language: String) = MovieEntity(
39             id = id,
40             title = title,
41             overview = overview,
42             posterPath = posterPath,
43             backdropPath = backdropPath,
44             voteAverage = voteAverage,
45             voteCount = voteCount,
46             releaseDate = releaseDate,
47             originalLanguage = originalLanguage,
48             popularity = popularity,
49             category = category,
50             language = language
51         )
52
53         private fun getMovies(
54             category: String,
55             language: String,
56             forceRefresh: Boolean,
57             apiCall: suspend () -> Response<MovieResponse>
58         ): Flow<ApiResponse<List<MovieEntity>>> = flow {
59             emit(ApiResponse.Loading)
60
61             if (!forceRefresh && isCacheValid(category,
62                 language)) {
63                 val cached =
64                 movieDao.getMoviesByCategoryAndLanguage(category,
65                     language).first()
66                 if (cached.isNotEmpty()) {
67                     emit(ApiResponse.Success(cached))
68                     return@flow
69                 }
70             }
71         }

```

```

63         }
64
65         try {
66             val response = apiCall()
67             when {
68                 response.isSuccessful -> {
69                     val results =
70 response.body()?.results
71                     if (!results.isNullOrEmpty()) {
72                         val entities = results.map {
73 it.toEntity(category, language) }
74
75 movieDao.deleteByCategoryAndLanguage(category, language)
76 movieDao.insertAll(entities)
77 val data =
78 movieDao.getMoviesByCategoryAndLanguage(category,
79 language).first()
80 emit(ApiResponse.Success(data))
81 } else {
82     serveStaleOnError(category,
83 language, ErrorType.NO_RESULTS)
84 }
85 }
86 else -> {
87     serveStaleOnError(category,
88 language, ErrorType.SERVER_OR_CLIENT_ERROR)
89 }
90 }
91 } catch (e: UnknownHostException) {
92     Timber.e(e, "[\$category] No Internet
93 Connection")
94     serveStaleOnError(category, language,
95 ErrorType.NO_INTERNET)
96 } catch (e: SocketTimeoutException) {
97     Timber.e(e, "[\$category] Connection timed
98 out")
99     serveStaleOnError(category, language,
100 ErrorType.TIMEOUT)
101 } catch (e: Exception) {
102     Timber.e(e, "[\$category] Unexpected error")
103     serveStaleOnError(category, language,
104 ErrorType.UNKNOWN)
105 }
106 }

```



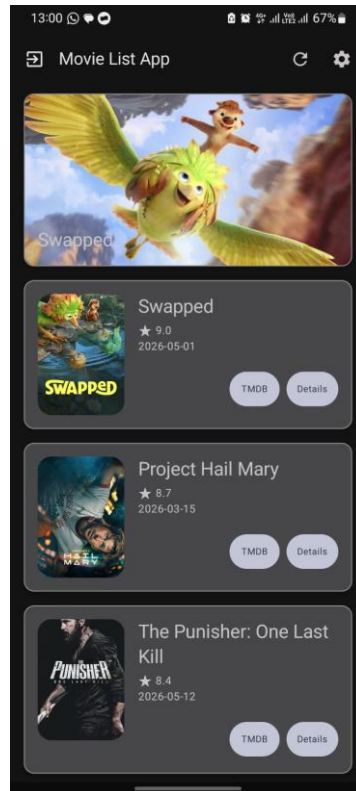
```

96     private suspend fun
FlowCollector<ApiResponse<List<MovieEntity>>>>.serveStale
OnError(
97         category: String,
98         language: String,
99         errorType: ErrorType
100     ) {
101         val stale =
movieDao.getMoviesByCategoryAndLanguage(category,
language).first()
102         if (stale.isNotEmpty()) {
103             emit(ApiResponse.Success(data = stale))
104         } else {
105             val anyLanguage =
movieDao.getAnyMoviesByCategory(category).first()
106             if (anyLanguage.isNotEmpty()) {
107                 emit(ApiResponse.Success(data =
anyLanguage))
108             } else {
109                 emit(ApiResponse.Error(errorType))
110             }
111         }
112     }
113
114     override fun getPopularMovies(language: String,
forceRefresh: Boolean):
Flow<ApiResponse<List<MovieEntity>>>> =
115         getMovies("popular", language, forceRefresh) {
116             apiService.getPopularMovies(language =
language)
117         }
118
119     override fun getNowPlayingMovies(language: String,
forceRefresh: Boolean):
Flow<ApiResponse<List<MovieEntity>>>> =
120         getMovies("now_playing", language, forceRefresh)
{
121             apiService.getNowPlayingMovies(language =
language)
122         }
123
124     override fun getTopRatedMovies(language: String,
forceRefresh: Boolean):
Flow<ApiResponse<List<MovieEntity>>>> =
125         getMovies("top Rated", language, forceRefresh) {

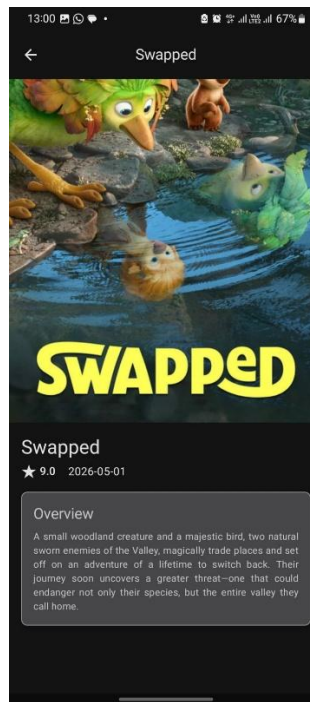
```

126	<code>apiService.getTopRatedMovies(language =</code>
127	<code>language)</code>
128	<code>}</code>

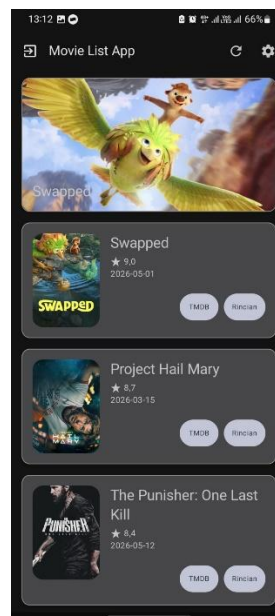
B. Output Program



Gambar 1. Screenshot Hasil Homepage (en)



Gambar 2. Screenshot Hasil Detail Page (en)



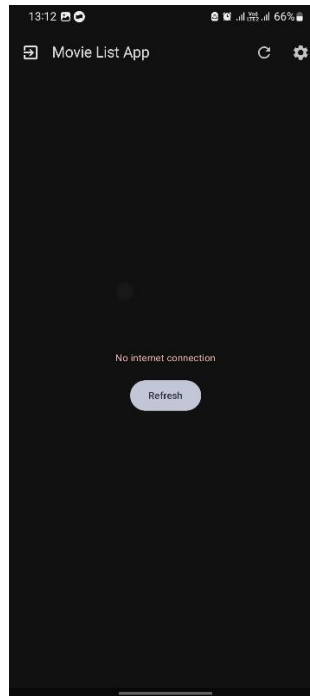
Gambar 3. Screenshot Hasil Homepage (id)



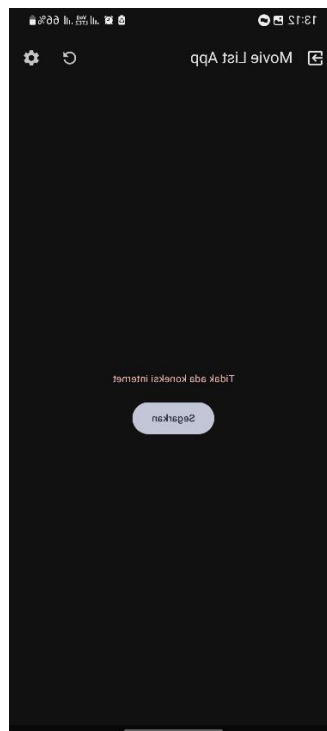
Gambar 4. Screenshot Hasil Movie List Now Playing (id)



Gambar 5. Screenshot Hasil Detail Page (id)



Gambar 6. Screenshot Hasil Movie List No Internet (en)



Gambar 7. Screenshot Hasil Movie List No Internet (id)

C. Pembahasan

ScrollableUiState.Kt, ErrorMessage.Kt & ScrollableViewModel.Kt

ScrollableUiState.Kt merupakan data class StateFlow yang menyimpan dan menginisialisasikan data berupa state atau kondisi UI yang terdapat pada aplikasi. State yang disimpan pada data class tersebut adalah data movies yang diambil dari MovieEntity.kt, state loading, state pesan error yang ditentukan dari interface tertutup (sealed) ErrorMessage.kt dengan dua state berupa ada yang ditentukan dari “ErrorMessageRes” dan tidak ada, indeks item yang dipilih, serta konfigurasi bahasa dan locale.

ScrollableViewModel.Kt merupakan class yang berfungsi untuk mengelola logika UI aplikasi dengan menerapkan arsitektur Model-View-ViewModel yang memisahkan data, logika bisnis, dan logika UI. Pada inisialisasi ViewModel, data-data aplikasi seperti konfigurasi bahasa dan locale serta list movies akan diinisialisasi dan di-load serta dikoleksikan ke dalam DataStore. Pada inisialisasi, state aplikasi sedang loading.

Method “loadMovies()” merupakan helper method yang digunakan pada saat inisialisasi dan pada saat refresh data pada method “refresh()” yang meng-observe dan meng-update data yang telah di-fetch dari API dan disimpan ke dalam repository berupa data movie populer. Method ini memberitahu aplikasi akan state dari API response serta bagaimana UI akan di-update berdasarkan API response. Jika sedang loading, tidak usah muat error message dan set state loading, jika sukses update data pada UI, jika gagal muat error message ke UI.

ScrollableViewModel.Kt memiliki parameter “initialLocale” bertipe data String yang memuat locale Aplikasi serta DataStore “preferencesRepository” dan data dari repository database “MovieRepository” yang nantinya akan diinisialisasi melalui ViewModelFactory. Selain itu, ada dua method khusus untuk tombol detail dan intent yang mengirimkan log ke Logcat serta terdapat log yang dikirimkan pada saat inisialisasi aplikasi dan log data yang dipilih ke Logcat.

Method “updateCurrentItem()” berfungsi untuk memperbarui indeks list item yang dipilih pada ScrollableUiState.Kt yang nantinya akan digunakan untuk melakukan navigasi ke Detail Page sesuai dengan indeks item yang dipilih. Method “updateLocale()” berfungsi untuk mengatur locale atau konfigurasi bahasa aplikasi dengan memperbarui locale yang nantinya disimpan ke DataStore untuk mengubah konfigurasi bahasa sesuai parameter.

MainActivity.Kt, ScrollableApplication.Kt & ScrollableApp.Kt

MainActivity.Kt berfungsi sebagai Activity utama dari Aplikasi Android dengan konfigurasi edge to edge, tema aplikasi menyesuaikan tema sistem Android, dan layout yang diambil dari method composable “ScrollableApp()” di file ScrollableApp.Kt. Pada file ScrollableApp.Kt, terdapat Class yang menyimpan tipe data enum dengan nama “ScrollableScreen” yang berisikan tiga konstanta yang dipakai untuk navigasi, yaitu “Home”, “Details”, dan “Settings”. ScrollableApplication.Kt merupakan Application Class yang berfungsi untuk melakukan inisialisasi global database serta DataStore dan “MovieRepository” dengan menerapkan Singleton Pattern. Selain itu, “onCreate()” digunakan untuk meng-inject dependensi Timber Log.

Method composable “ScrollableApp()” berfungsi untuk menampilkan layout utama Aplikasi pada Activity serta navigasi aplikasi per layar screen. Sebelumnya, terdapat inisialisasi locale dan repository dari database dan DataStore yang dilakukan menggunakan ViewModelFactory. Navigasi dilakukan dengan “NavHost” dan “NavController” dengan layar default Homepage dan tiga rute navigasi yang dikoneksikan melalui method navigator “composable()”. Pada rute Homepage, terdapat inisialisasi composable “HomeScreen()” yang menginisialisasikan UI State dari ScrollableUiState, logika tombol detail yang bernavigasi ke screen Detail Page sesuai nilai indeks, logika tombol intent yang menjalankan explicit intent ke TMDB Page, logika tombol setting yang bernavigasi ke screen Setting Page, dan logika tombol exit yang menutup aplikasi. Tombol intent dan detail terdapat panggilan ke ViewModel untuk logging ke Logcat.

Pada rute Detail Page, nilai indeks item akan diperbarui secara asynchronous, kemudian menginisialisasikan composable “DetailScreen()” dengan parameter “item” yang menentukan item mana yang dimuat dari data, “modifier” sebagai modifier layout, dan “onBackClick” sebagai logika tombol back. Pada rute Setting Page, terdapat inisialisasi composable “SettingScreen()” dengan parameter “modifier” sebagai modifier layout, “onBackClick” sebagai logika tombol back, “onLocaleChange” sebagai logika konfigurasi bahasa yang memanggil method pada ViewModel, dan “selectedLocale” yang menyimpan State Locale ke ScrollableUiState.

SettingScreen.Kt Compose

Method Composable “SettingScreen()” berfungsi untuk memuat layout UI pada Setting Page dengan komponen “Scaffold” untuk memuat kerangka UI yang terdiri dari Topbar orientasi tengah dengan title atau judul dan ikon navigasi yang diambil dari Material 3 yang memanggil method “onBackClick” jika ditekan. Layout dibawah Topbar terdiri dari “Column” yang memuat kolom dengan padding yang menyesuaikan komponen “Scaffold” dan jarak keseluruhan 16dp dan menyimpan UI serta dapat melakukan scroll vertikal dan menyimpan posisi scroll.

Di dalam komponen “Column”, terdapat komponen “Text” yang memuat teks pengaturan/setting dengan gaya dari Material 3 Headline dan padding vertikal 8dp. Kemudian, terdapat tiga komponen yang berfungsi untuk memuat masing-masing UI Radio Button, yaitu komponen “Row” yang memuat baris dengan orientasi tengah vertikal yang didalamnya terdapat komponen “RadioButton” yang terpilih jika nilai “selectedLocale” sesuai dengan Locale Sistem dan panggilan method untuk mengubah Locale, serta komponen “Text” yang mengasih keterangan pada tiap komponen “RadioButton”. Method preview “SettingScreenLayout()” berfungsi untuk memuat UI dari composable yang telah dibuat tanpa harus menjalankan aplikasi.

DetailScreen.Kt Compose

Method Composable “DetailScreen()” berfungsi untuk memuat layout UI pada Detail Page dengan komponen “Scaffold” untuk memuat kerangka UI yang terdiri dari Topbar orientasi tengah dengan title dengan komponen “Text” dan ikon navigasi dari Material 3 yang memanggil method “onBackClick” jika ditekan. Layout dibawahnya terdiri dari komponen kolom “Column” scrollable dengan padding menyesuaikan kerangka.

Di dalam komponen “Column”, terdapat komponen gambar “AsyncImage” yang mengambil sumber gambar dari data API fetch dengan lebar memenuhi parent dan tinggi 450dp serta terdapat gambar yang dimuat saat loading/placeholder dan error/fallback. Di bawahnya, terdapat komponen “Column” dengan padding keseluruhan 16dp yang memuat komponen “Text” title dan subtitle dengan gaya teks Material 3 Headline. Dibawahnya, terdapat komponen view baris “Row” yang memuat ikon bintang serta teks rata-rata vote dan tanggal rilis dengan fallback text jika nilai pada data tidak ada.

Di bawahnya, terdapat komponen “Card” yang memuat suatu tempat dengan padding atas 16dp, elevasi 4dp, bentuk pojok melengkung sebesar 8dp, outline setebal 1dp dengan gaya Material 3 outline. Di dalam komponen “Card”, terdapat komponen “Column” dengan padding 16dp yang memuat komponen “Text” overview title dengan gaya Material 3 Title dan “Text” detail overview dengan gaya Material 3 Body dengan fallback text jika nilai pada data tidak ada.

HomeScreen.Kt Compose

Method Composable “HomeAppBar()” berfungsi untuk memuat layout Topbar UI yang terdiri dari Topbar biasa yang terdiri dari sub komponen title, ikon navigasi exit, dan ikon navigasi actions yang digunakan untuk refresh berpindah ke Setting Page. Method Composable “HomeScreen()” berfungsi untuk memuat layout UI Homepage menggunakan komponen “Scaffold” sebagai kerangka yang terdiri dari Topbar yang diambil dari method composable “HomeAppBar()” dengan parameter “OnSettingClick” untuk berpindah ke setting dan parameter “OnExit” untuk keluar dari aplikasi.

Isi dari “HomeScreen()” secara utama adalah kolom “Column” dengan padding yang menyesuaikan “Scaffold” yang akan menampilkan loading bar jika data kosong dan state aplikasi loading. di bawahnya, terdapat komponen “Box” dengan layout memenuhi view. View pada box ditentukan oleh state aplikasi. Jika loading, tampilkan loading spinner, jika gagal (ada error message) tampilkan pesan error di dalam “Text” serta tombol refresh dibawahnya.

Jika data berhasil di-load adalah komponen “LazyColumn” dengan padding yang terdapat di dalam view adalah 8dp pada bagian kiri dan kanan serta 16dp pada bagian bawah yang memuat UI scrollable yang hanya terlihat pada layar. “LazyColumn” digunakan untuk memuat data-data yang dimuat pada Carousel yang didefinisikan pada composable “ItemCarousel()” serta logika tombol detail. Selain itu, juga terdapat pemuatan data-data berbasis indeks ke item list yang didefinisikan pada composable “ItemCard()” dengan logika tombol detail serta tombol dengan explicit intent.

Composable “ItemCard()” berfungsi untuk memuat UI item list yang terdiri dari komponen “Card” dengan padding 8dp, ukuran Material 3 Medium, elevasi 4dp, dan border atau outline setebal 1dp dengan warna Material 3 outline. Di dalamnya, terdapat komponen “Row” dengan padding keseluruhan 16dp yang berisikan komponen gambar “AsyncImage” yang mengambil sumbernya dari data API fetch, isi gambar memenuhi View, dan modifier tinggi 140dp dan lebar 100dp serta ukuran pojok melengkung sebesar 16dp serta terdapat gambar yang dimuat saat loading/placeholder dan error/fallback.

Di sebelah komponen “AsyncImage”, terdapat komponen “Column” dengan padding awal 16dp dan komponen mengambil setengah dari space parent View. Di dalamnya, terdapat komponen “Text” yang memuat teks title dengan gaya material 3 Headline, ikon bintang dan teks rata-rata vote dan tanggal rilis dibawahnya dengan gaya material 3 body. Di bawahnya, terdapat komponen “Row” dengan padding atas 16dp dan orientasi child View berada di akhir. Di dalamnya, terdapat komponen tombol “Button” dengan tombol pertama memiliki padding horizontal 12dp dan kedua teks pada tombol memiliki gaya Material 3 body. Kedua tombol dipisah dengan komponen “Spacer” dengan jarak 8dp.

Composable “CarouselCard()” berfungsi untuk memuat UI isi Carousel yang terdiri dari komponen “Card” dengan padding 8dp, ukuran Material 3 Medium, elevasi 4dp, border atau outline setebal 1dp dengan warna Material 3 outline, warna Kartu atau “Card” berupa Material 3 Surface Container, dan memanggil “onDetailClick” jika ditekan. Di dalamnya, terdapat komponen “Box” dengan tinggi 200dp. Komponen “AsyncImage” di dalam “Row” mengambil data remote, isi gambar memotong dirinya jika memenuhi View, dan modifier lengkungan sebesar 16dp serta gambar placeholder saat loading dan gambar fallback jika error. Didalamnya, terdapat komponen “Box” yang terletak di bawah kiri dengan padding 16dp yang memuat teks title dengan gaya Material 3 Headline.

Composable “ItemCarousel()” berfungsi untuk memuat UI Carousel melalui komponen “LazyRow” yang menyimpan scroll state dan berhenti di item terdekat dalam scrolling. Di dalamnya, terdapat pemuatan data-data berdasarkan indeks ke dalam composable “CarouselCard()” beserta logika klik navigasi ke detail serta lebar child Composable mengambil penuh lebar View.

AppPreferencesRepository.Kt

AppPreferencesRepository.Kt adalah class yang menggunakan DataStore untuk menyimpan preferensi yang digunakan pada aplikasi menggunakan preference DataStore sebagai pengganti modern dari SharedPreferences. Preferensi yang disimpan yaitu preferensi locale. Instance DataStore yang dibuat hanya sekali dan dinamakan dengan “app_preferences” yang menyimpan nilai key yang didefinisikan pada konstanta “KEY_LOCALE” yang menyimpan locale yang dipilih.

Class ini membaca tiap data sebagai Flow yang meng-observe perubahan data secara otomatis kemudian mengekstraknya dan disimpan ke dalam DataStore pada variabel immutable “selectedLocale”. Ekstraksi data dilakukan dengan error handling terlebih dahulu jika terdapat kesalahan saat membaca data. Selain itu, terdapat method “saveLocale” yang menyimpan data ke dalam DataStore yang berupa preferensi locale ke DataStore yang hanya bisa dijalankan sebagai coroutine.

MovieDao.Kt

MovieDao.Kt adalah Data Access Object (DAO) yang merupakan interface sebagai abstraksi tanpa harus berinteraksi ke layer data secara langsung yang dinotasikan dengan anotasi “@Dao”. Method “getMoviesByCategoryAndLanguage()” digunakan untuk mengambil data dari tabel Movies berdasarkan kategori dan bahasa yang diurutkan berdasarkan rata-rata suara menurun. Method “insertAll” berfungsi untuk memasukkan seluruh nilai ke dalam tabel. Method “deleteByCategoryAndLanguage” berfungsi untuk menghapus data Movies berdasarkan kategori dan bahasa.

Method “getLastCachedTime” berfungsi untuk mendapatkan waktu cache dari Movies berdasarkan kategori dan bahasa dengan hasil yang dibatasi 1. Method “getAnyMoviesByCategory” berfungsi untuk mendapatkan data Movies berdasarkan kategori yang diurutkan berdasarkan rata-rata suara menurun. Anotasi “@Query” menandakan method query untuk berinteraksi dengan database, sedangkan anotasi “@Insert” menandakan method insert untuk memasukkan data pada database yang mengganti data lama jika terjadi konflik. Syntax “suspend” membuat method hanya bisa dijalankan di coroutine.

MovieEntity.Kt

MovieEntity.Kt merupakan Class yang digunakan sebagai blueprint untuk melakukan mapping pada baris pada tabel Database ke Objek Kotlin dengan anotasi “@Entity” dengan keterangan nama tabel “movies”, dua primary key berupa kolom “id” dan “language” serta melakukan indexing pada kolom “category” dan “language”. Data yang akan di mapping ke database berupa “id”, “title”, “overview”, “posterPath”, “backdropPath”, “voteAverage”, “releaseDate”, “originalLanguage”, “popularity”, “category”, “language”, dan “cachedAt” dengan nilai default waktu sekarang dalam milidetik. Selain itu, terdapat variabel immutable “fullPosterUrl” sebagai helper untuk menampilkan URL Poster ke UI dan variabel immutable “fullBackdropUrl” sebagai helper untuk menampilkan URL backdrop poster ke UI.

MovieDatabase.Kt

MovieDatabase.Kt merupakan Class yang digunakan untuk instansiasi Room Database melalui ekstensi ke Class “RoomDatabase” serta anotasi “@Database” dengan entitas yang ditentukan oleh MovieEntity.Kt, versi database 1, dan tanpa ekspor skema Database. Terdapat abstract method “movieDao()” yang menghubungkan Database ke DAO. Kemudian, terdapat intansiasi satu Instance Database dengan anotasi “@Volatile” agar tidak terdapat pembuatan Instance Database lain pada thread lainnya. Method “getDatabase()” digunakan untuk menginstansiasi Room Database hanya dan jika hanya terdapat satu Database yang dimintakan oleh thread.

TmdbApiService.Kt

TmdbApiService.Kt merupakan interface yang berfungsi untuk melakukan remote API fetch dari TMDB. Ketiga method terdapat syntax “suspend” agar hanya bisa dijalankan pada coroutine. Selain itu, terdapat anotasi “@GET” yang mengirimkan HTTP GET request pada page popular untuk method “getPopularMovies()”, page now playing untuk method “getNowPlayingMovies()”, dan page top rated untuk method “getTopRatedMovies()” dengan tiap query parameter halaman atau page serta bahasa.

MovieDto.Kt & MovieResponse.Kt

MovieDto.Kt merupakan data class yang berfungsi sebagai Data Transfer Object (DTO) yang berfungsi untuk transfer data pada layer yang berbeda tanpa harus melakukan interaksi secara langsung. Class ini memiliki anotasi “@Serializable” yang digunakan untuk melakukan konversi data dengan format yang berbeda-beda. Class ini memuat data berupa “id”, “title”, “overview”, dan “popularity”, serta data dengan anotasi “@SerializedName” yang melakukan formatting dari data awal ke objek kotlin yang digunakan pada “posterPath”, “backdropPath”, “voteAverage”, “voteCount”, “releaseDate”, “genreIds”, dan “originalLanguage”. MovieResponse.Kt merupakan data class serializable yang berfungsi sebagai wrapper untuk MovieDto.Kt yang memuat page apa yang di load serta isi dari page tersebut serta total page & result yang di load.

ApiResponse.Kt & ErrorType.Kt

ApiResponse.Kt merupakan interface yang digunakan oleh data yang telah dipanggil melalui API Call untuk mengetahui state atau kondisi apakah proses API Call berhasil, gagal, atau sedang dimuat atau loading. Pada state success, data dari API Call akan dimuat melalui perantara interface. Pada state gagal, data tidak disimpan dan emit error type yang ditentukan dari enum class ErrorType.Kt untuk memberitahu jenis error apakah yang sedang terjadi. Pada state loading, data tidak dimuat hingga loading selesai.

NetworkModule.Kt

NetworkModule.Kt merupakan suatu objek Singleton yang mengatur dan memuat satu instance dari konfigurasi network aplikasi seperti Retrofit dan OkHttp. Objek ini memuat konstanta base url TMDB serta melakukan konfigurasi JSON yang mengabaikan nilai key tidak diketahui, memberikan nilai default jika terdapat null pada JSON, serta bagaimana string JSON diformat jika terdapat ketidaksesuaian formatting.

Objek ini memiliki dua interceptor berupa “loggingInterceptor” yang memuat detail isi dari API Call ke dalam log pada debug mode, serta “authInterceptor” yang memuat API Key pada tiap request yang akan dilakukan. Selain itu, terdapat “okHttpClient” yang mengatur koneksi HTTP dengan menjalankan kedua interceptor tersebut serta memberikan timeout 30 detik untuk koneksi, read, dan write.

Instansiasi Retrofit dilakukan dengan menghubungkan base url dengan client yang dikonfigurasi melalui “OkHttpClient” lalu menggunakan Converter Factory untuk mengkonversikan JSON ke objek Kotlin sesuai dengan konfigurasi yang telah diberikan. Kemudian, API Service TMDB diinstansiasi dari konfigurasi Retrofit sebelumnya.

IMovieRepository.Kt & MovieRepository.Kt

IMovieRepository.Kt merupakan interface yang akan diterapkan oleh MovieRepository.Kt sebagai abstraksi yang nantinya akan di-inject melalui Application Class. Interface ini memiliki 3 method yang akan di-override, yaitu “getPopularMovies” untuk mengembalikan data movie populer dari API fetch, “getNowPlayingMovies” untuk mengembalikan data movie yang sedang tayang dari API fetch, dan “getTopRatedMovies” untuk mengembalikan data movie dengan rating tertinggi dari API fetch.

Class `MovieRepository.Kt` memiliki parameter `API Service` untuk mengambil data dari remote API fetch serta `DAO` untuk `CRUD` database. Selain method yang di-override dari interface, Class ini juga terdapat beberapa helper method seperti `“isCacheValid()”` yang berupa coroutine yang menentukan validitas cache (< 30 menit). Kemudian, terdapat extension function `“toEntity()”` yang melakukan mapping data dari `DTO` ke kolom-kolom Database.

Method helper `“getMovies()”` digunakan untuk fetch data movies dari API maupun cached data. Jika API response sukses (dan terdapat data response), lakukan mapping data ke database lokal, update data, fetch dan notify success. Jika tidak, notify error berdasarkan jenis error yang telah terjadi atau fetch data lama/stale data melalui Extension function `“serveStaleOrError()”`.

Tautan GIT

Berikut adalah tautan GIT untuk Praktikum Pemrograman Mobile

<https://github.com/Arz-zrs/Pemrograman-Mobile>